Experiment No. 2

Title: To implement operations on individual image pixels

Apparatus use:

A PC with Python 3.7.8 and OpenCV software Jupyter notebook

Theory :

Point processing is used to transform an image by operating on individual pixel.

If array A represents an input image, then an output array B is produced by a transformation

$$B[x, y] = T [A[x, y]]$$

1. **Read specific pixel value**

```
import cv2
src=cv2.imread('Lenna.jpg')
print(src[100,100][0])
print(src[100,100][1])
print(src[100,100][2])
```

Output:

```
In [15]: import cv2
         src=cv2.imread('Lenna.jpg')
         print(src[100,100][0])
         print(src[100,100][1])
         print(src[100,100][2])

         163
         112
         179
```

2. **Display separate R,G,B channels:**

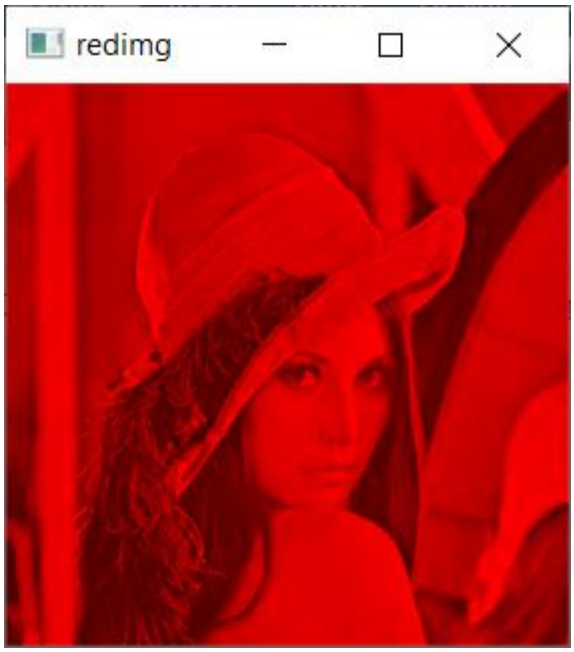
- **Red channel**

Code:

```
import cv2
```

```
path=r'Lenna.jpg'
src=cv2.imread(path)
src[:,:,0]=0
src[:,:,1]=0
cv2.imshow("redimg",src)
cv2.waitKey(0)
```

Output:



- **Blue Channel**

  Code:

```
import cv2
path=r'Lenna.jpg'
src=cv2.imread(path)
src[:,:,1]=0
src[:,:,2]=0
cv2.imshow("Blue_img",src)
cv2.waitKey(0)
```

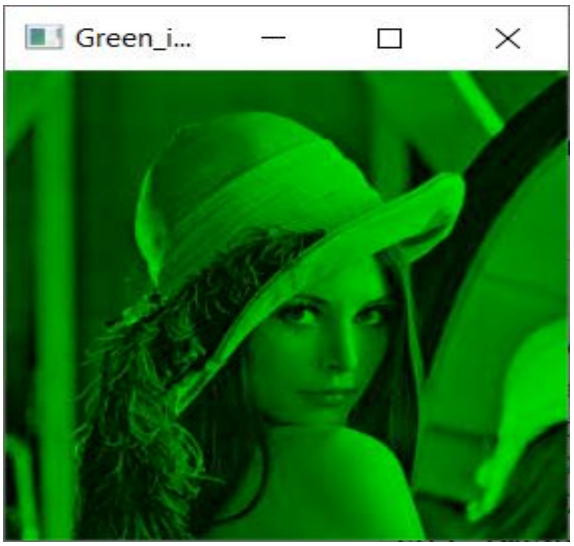
Output:

- **Green Channel**

Code:

```
import cv2
path=r'Lenna.jpg'
src=cv2.imread(path)
src[:,:,0]=0
src[:,:,2]=0
cv2.imshow("Green_img",src)
cv2.waitKey(0)
```

Output:



3. Covert image in different color spaces

- **Negative Image:**

Code:

```
import cv2
src=cv2.imread('Lenna.jpg')
max1=src.max()
negimg=max1-src
cv2.imshow("neg_img",negimg)
cv2.waitKey(0)
```

Output:

- **Gray image**

Code:

```
import cv2
src=cv2.imread('Lenna.jpg')
gray_image = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)
cv2.imshow("Gray image",gray_image)
cv2.waitKey(0)
```

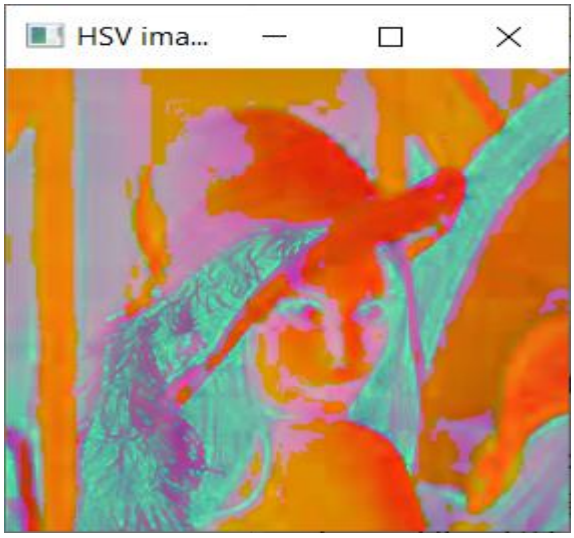
Output:

- **HSV image**

Code:

```
import cv2
src=cv2.imread('Lenna.jpg')
hsv_image = cv2.cvtColor(src,cv2.COLOR_BGR2HSV)
cv2.imshow("HSV image",hsv_image)
cv2.waitKey(0)
```

Output:

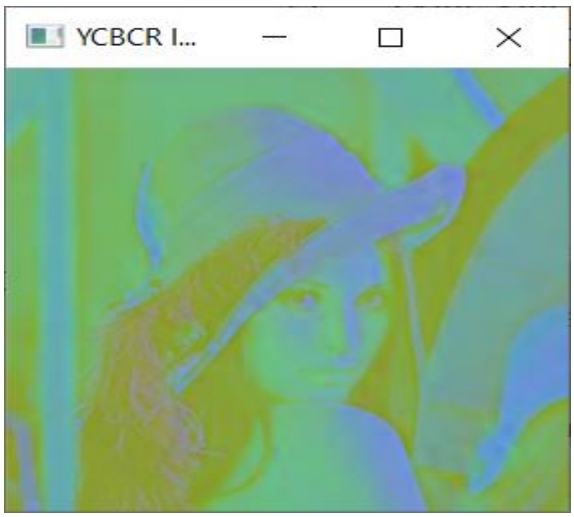


- **YCBCR image**

Code:

```
import cv2
src=cv2.imread('Lenna.jpg')
ycbcr_img= cv2.cvtColor(src,cv2.COLOR_BGR2YCR_CB)
cv2.imshow(" YCBCR image",ycbcr_image)
cv2.waitKey(0)
```

Output:



Viva Questions:

- Write a Python program to read a grayscale image and display it.
- Implement image negative transformation using pixel-wise operation.
- Increase the brightness of an image by 50 units and display the result.
- Apply binary thresholding with a threshold value of 127.
- Perform contrast stretching on an input image using min-max normalization.
- Compare original and negative images using matplotlib.
- Combine two pixel-wise operations (e.g., brightness increase + thresholding) and show the final result.
- Display histograms of original and contrast-stretched images. Discuss the difference.
- Write a program that takes user input to choose the type of transformation (negative, brightness, threshold, etc.).
- Implement pixel-wise operations for a color image (perform transformation on each RGB channel).
- Evaluate how changing the threshold value affects the segmentation result in thresholding.
- What will be the output of applying `img = 255 - img` on a white image?
- Observe the output of log transformation: What happens to the dark regions?
- If you apply a threshold of 100 to an image, what part of the image becomes white?
- Compare the image output for gamma values <1 and >1. What differences do you see?
- How does contrast stretching enhance details in low-contrast images?